

Compressed by Discovery: Snapped Symbolic-Regression Primitives as Realizable, Verifiable Function Units on DSP-less FPGAs

Daniyel Yaacov Bilal, Chokmah LLC, chokmah-dyb@pm.me, ORCID: 0000-0002-9040-6914

v1.0 draft — DOI: *to be minted on Zenodo deposit*

Technical note accompanying *Valid and False Snapping in EML Expression Trees: The Basin Selection Problem* (the "companion paper"; v2.3 at time of writing) and its basin-selection warm-start note (v1.0, DOI 10.5281/zenodo.20671272). This note reports the Track C hardware study of the repository `eml-ice40-cybenko`. Companion artifacts: `m1p/` (the MLP and ROM baselines), `hardware/` (the snapped-EML units), `results/m1p_pareto.csv` (synthesis sweep), `notes/figure_symbolic_vs_m1p.png`.

Abstract

Deploying a *learned* scalar function on a small FPGA forces a choice between two unattractive corners. A direct lookup ROM, `ROM[x] = quantize(f(x))`, is bit-exact to the true function's fixed-point image and exhaustively verifiable, but its size is set by the input wordlength and grows as the product of per-input domains. A quantized MLP is compact in parameters, but on fabric without hard multipliers every multiply maps into LUTs and carry chains, and it is exact only to its own trained approximation floor. We show that a third option occupies the realizable gap between them: an elementwise-ML (EML) expression tree recovered by symbolic regression and then *argmax-snapped* to a fixed-function datapath. After snapping, the soft simplex mixing vanishes and each gate is `exp(a) - ln(b)` with inputs routed from `{1, x, child}` — a tiny fixed-point datapath.

On a Lattice iCE40 (no hard multipliers), the two canonical recovered primitives — `exp eml(x, 1)` and `ln eml(1, eml(eml(1, x), 1))` — are simultaneously **smaller** in logic cells than every quantized MLP we train (width $H \in \{4, 8, 16, 32\}$, depth $\in \{1, 2\}$, best of 8 seeds) **and more accurate**, and they are bit-exact to the true function's fixed-point image, confirmed 256/256 on a physical iCEstick. Against the strongest classical baseline, a direct ROM, the picture sharpens: for `exp` the ROM *ties* the symbolic unit (8 EBR + 97 LC, same accuracy), but for `ln` it **collapses** — the 22-bit Q10.12 input needs ≈ 512 block RAMs, infeasible on any iCE40 — while the symbolic `ln` unit fits an HX8K (1964 LC + 13 EBR) via leading-one range reduction. The unifying observation is that **the snapped symbolic primitive is a compressed ROM**: range reduction and interpolated LUTs shrink an unrealizable table to a few EBR. Its advantage is therefore smallest for cheap single-input cases (where the ROM ties it) and grows with input wordlength and arity. We argue this makes symbolic-regression discovery a practical route to compact, *exhaustively verifiable* function units for safety- and certification-sensitive edge inference, and we report both the area-accuracy Pareto frontier and on-silicon confirmation through one end-to-end pipeline.

1. Introduction

A recurring problem in edge inference is realizing a learned scalar nonlinearity — an activation, a calibration curve, a recovered physical law — as compact, trustworthy hardware on a small FPGA. Two answers are obvious. Tabulate it: store `quantize(f(x))` in a ROM addressed by the input code. Or learn it: train a small quantized MLP and synthesize the MACs. Each has a well-known failure. The ROM is exact and trivially verifiable but its size is set by the input wordlength, not the function's complexity, so it explodes with precision and with the number of inputs. The MLP is parameter-compact but, on fabric without DSP blocks, every multiply costs LUTs and carry chains, and — more fundamentally — it is correct only to a training-dependent approximation floor, which is a weak guarantee for certification-sensitive use.

The companion paper and its warm-start note studied a different object: EML (elementwise-ML) expression trees, in which a single binary operator `eml(a,b) = exp(a) - ln(b)` is composed into a tree whose input routing is learned by softmax selectors and then *snapped* (argmax-committed) to a discrete symbolic form. That work was about *recovering the right form* (the basin-selection problem). This note asks the downstream hardware question: once snapped, what kind of function unit is the result, and how does it compare — on real silicon — to the two obvious baselines?

The answer is a clean three-corner story. The snapped EML unit dominates the quantized MLP on area and accuracy at once on iCE40; it ties the direct ROM for a cheap single-input function (exp) and decisively beats it for a wide-input one (ln), where the ROM is physically unrealizable. The single sentence that unifies the data is that **the snapped symbolic primitive is a compressed ROM** — and the compression is exactly what buys realizability where direct tabulation fails. All claims run through one golden-model → Icarus → yosys/nextpnr → iceprog pipeline, with every RTL variant verified bit-exact 256/256 in simulation and the symbolic exp unit confirmed on a physical iCEstick.

2. Background: snapped EML trees as fixed-function datapaths

A balanced EML tree mixes its candidate inputs `{1, x, child}` through per-selector softmaxes during training. The companion paper drives commitment with a three-phase schedule (task loss; entropy ramp; temperature anneal), and accepts a run as *valid* only if every selector snaps at max-weight ≥ 0.9 **and** the post-snap MAE on the grid is < 0.01 . Under that criterion all 152 valid forms in the released CSVs canonicalize to exactly two designs: `exp eml(x,1)` at depth 2 and `ln eml(1, eml(eml(1,x),1))` at depth 4.

After snapping there is no softmax left. Each gate is the literal arithmetic `exp(a) - ln(b)`, its inputs are fixed wires drawn from `{1, x, child}`, and off-path candidates constant-fold away at generation time. The result is a small fixed-point datapath, not a network: exp is one interpolated `exp` LUT; ln is one `ln` LUT preceded by a leading-one range reduction `ln(m·2k) = ln_lut[m] + k·ln2`. Track C compiles these to iCE40 RTL (`hardware/`), and the integer Python model `hardware/fixed_point.py` mirrors the Verilog operation-for-operation, serving as the golden reference.

3. Method: one pipeline, three baselines

All three function-unit families are measured the same way, so the comparison is apples-to-apples.

- **Same targets / domains.** exp on $[-2, 2]$, ln on $[0.1, 10]$ — the intervals the released CSV solutions were recovered on.
- **Same fixed-point formats.** exp Q8.8 (16-bit), ln Q10.12 (22-bit), matching the snapped-EML units in `hardware/HARDWARE.md`.

- **Same streaming wrapper.** Every core is wrapped by `hardware/verilog_gen_stream.py` (byte-serial, 19 pins) and synthesized for the UP5K sg48 — the variant the symbolic stream rows were measured on.
- **Same golden-model contract.** `mlp/fixed_point_mlp.py::QuantMLP` and `hardware/fixed_point.py` are integer-only forward passes that mirror the emitted Verilog operation-for-operation. `mlp/sim_check_mlp.py` confirms all MLP cells (bare core + stream wrapper) are bit-exact 256/256 under Icarus; the symbolic and ROM units likewise.

The three baselines:

1. **Snapped EML units** (`hardware/`): the two canonical recovered forms.
2. **Quantized MLP** (`mlp/`): textbook $1 \rightarrow H \rightarrow \dots \rightarrow H \rightarrow 1$, ReLU on hidden layers, one pipeline stage per Linear layer, weights as RTL constants. Each cell is trained from 8 independent seeds; the best by float-grid max-error supplies the RTL/area number, and we report mean $\pm$ std across seeds so the accuracy axis reflects capacity rather than a single seed's luck.
3. **Direct ROM** (`mlp/rom_baseline.py`): `ROM[x_in] = quantize(f(x_in))`, addressed by the input code, same wrapper and pipeline.

On quantization fairness. The MLPs are post-training quantized (train in float, then cast to the Q-format).

Quantization-aware training (QAT) would be a fairer baseline only if quantization were the bottleneck — and it is not: the fixed-point max-error tracks the float max-error to well within the symbolic-MLP gap at every cell. QAT can recover only the (small) PTQ residual; even driving it to zero leaves the best MLP at its *float* ceiling (exp 0.033, ln 0.059), which is still above the symbolic unit's *fixed-point* accuracy (exp 0.015, ln 0.0011). The separation is set by the ReLU network's piecewise-linear approximation capacity, which QAT does not change. We therefore report PTQ numbers and treat the MLP float ceiling as its fair upper bound.

4. Results

4.1 Accuracy: quantization is not the bottleneck

Fixed-point max-error tracks the float approximation error at both formats, so the MLP's accuracy ceiling is set by model capacity and training, not quantization. With best-of-8 selection the per-cell error is monotone in capacity within each depth. But even the largest, best-of-8 networks — exp h32 d2 (0.048 fp max-err) and ln h32 d2 (0.026) — remain worse than the snapped-EML units (exp 0.015, ln 0.0011). The symbolic primitive expresses the function exactly up to LUT interpolation; the ReLU MLP pays a piecewise-linear approximation tax.

4.2 Area and the Pareto frontier

Logic cells and routed Fmax are from yosys `synth_ice40` → nextpnr-ice40 on the byte-serial stream wrapper, targeting iCE40 UP5K sg48 (5280 LCs). `fp max-err` is the fixed-point error vs the true function (best of 8 seeds); `float  $\pm$  std` is the float-grid max-error across the 8 seeds. Device references: HX1K 1280, UP5K 5280, HX8K 7680 LCs. **LCs** is the nextpnr `ICESTORM_LC` count, printed even when placement overflows the device. A `†` marks cells whose synthesis did not finish in the 1-hour cap; for these we report a lower bound (the largest measured same-function cell — strictly larger, and losing on accuracy regardless). Every row is bit-exact 256/256 in simulation.

exp on $[-2, 2]$ (Q8.8):

unit	LCs	fits	Fmax (MHz)	fp max-err	float $\pm$ std (N=8)
snapped EML <code>eml(x, 1)</code>	316	HX1K	28.3	0.0150	—
MLP h4 d1	1423	UP5K	15.2	0.4016	1.089 $\pm$ 0.935
MLP h8 d1	2640	UP5K	14.5	0.1438	0.256 $\pm$ 0.124
MLP h16 d1	4808	UP5K	13.0	0.1282	0.144 $\pm$ 0.065
MLP h32 d1	8673	none	—	0.0930	0.108 $\pm$ 0.039
MLP h4 d2	3587	UP5K	15.4	0.2298	0.739 $\pm$ 0.722
MLP h8 d2	11156	none	—	0.0831	0.124 $\pm$ 0.045
MLP h16 d2	32276	none	—	0.0930	0.066 $\pm$ 0.031
MLP h32 d2	≥ 32276 †	none	—	0.0480	0.033 $\pm$ 0.011

In on [0.1, 10] (Q10.12):

unit	LCs	fits	Fmax (MHz)	fp max-err	float $\pm$ std (N=8)
snapped EML <code>eml(1, eml(eml(1, x), 1))</code>	1964	HX8K	16.7	0.00107	—
MLP h4 d1	3066	UP5K	12.5	0.4549	1.338 $\pm$ 0.726
MLP h8 d1	4933	UP5K	13.4	0.2105	0.614 $\pm$ 0.307
MLP h16 d1	9454	none	—	0.1461	0.292 $\pm$ 0.107
MLP h32 d1	18468	none	—	0.0506	0.085 $\pm$ 0.023
MLP h4 d2	7281	HX8K	—	0.2840	0.634 $\pm$ 0.258
MLP h8 d2	23319	none	—	0.1510	0.540 $\pm$ 0.657
MLP h16 d2	≥ 23319 †	none	—	0.0658	0.243 $\pm$ 0.261
MLP h32 d2	≥ 23319 †	none	—	0.0265	0.059 $\pm$ 0.029

The headline figure ([notes/figure_symbolic_vs_mlp.png](#)) plots logic cells vs fixed-point max-error per function. The two snapped-EML units dominate the plane — lower area **and** lower error than the entire MLP family. The cleanest statement is on the accuracy axis alone: no MLP cell, at any width, depth, or seed in the sweep, reaches the symbolic unit's fixed-point accuracy (best MLP exp 0.048 vs 0.015; best MLP ln 0.026 vs 0.001). The mechanism is concrete: iCE40 has no DSP blocks, so every signed multiply is built from fabric LUTs and carry chains, and area scales with the MAC count ($\approx H$ for depth 1, $\approx H^2$ for depth 2). MLPs blow past the device before reaching the symbolic accuracy. The snapped-EML unit's cost is just two small interpolated LUTs per gate.

4.3 The direct-ROM baseline and the compression thesis

The sharpest alternative to both is to skip arithmetic entirely: a scalar function of one fixed-point input is, in the limit, a lookup table. Like the symbolic unit, a direct ROM is bit-exact to the true function's fixed-point image and exhaustively verifiable — the table *is* the specification. Its size, however, is set by the input wordlength and the domain, not by the function's complexity.

function	unit	LCs	EBR	fp max-err	smallest iCE40
exp [-2,2] Q8.8	snapped EML <code>eml(x, 1)</code>	316	3	0.0150	HX1K
exp [-2,2] Q8.8	direct ROM (2048×16)	97	8	0.0149	HX1K
ln [0.1,10] Q10.12	snapped EML	1964	13	0.00107	HX8K
ln [0.1,10] Q10.12	direct ROM (65536×22)	—	512	0.00107	none

For **exp**, the ROM is a genuine competitor and we concede it: 2048 input codes × 16 b = 8 EBR + 97 LC at the same accuracy as the symbolic unit — both limited by the Q8.8 input resolution — at fewer LCs (it trades logic for block RAM). At this scale the symbolic exp unit's edge is not area but provenance and verifiability (§5); the ROM ties it.

For **ln**, the ROM **collapses**: the 22-bit Q10.12 input over [0.1, 10] needs ≈65 536 entries × 22 b ≈ 1.44 Mbit ≈ 512 block RAMs — 16× the 32 EBR on an HX8K, infeasible on any iCE40. The snapped-EML ln unit fits an HX8K precisely because leading-one range reduction is the *compression* of that table. The general statement: **the snapped-EML / interpolated-LUT unit is a compressed ROM** — exp shrinks a 2048-entry table to a 257-entry interpolated LUT (~16×); ln shrinks an unrealizable 512-EBR table to 13 EBR. For one narrow-input function the compression can be unnecessary (exp); its value appears exactly where the direct table fails — wide input wordlength (ln) and, decisively, **multiple inputs**, where a direct ROM grows as the *product* of per-input domains while the symbolic datapath does not. The ROM-collapse argument therefore bounds the claim rather than refuting it: symbolic ≈ ROM for cheap single-input cases, symbolic >> ROM as soon as input width or arity grows.

4.4 Arity: the ROM grows as a product, the symbolic unit as a sum

§4.3 ends on an assertion — that the symbolic unit's edge over the direct ROM *widens with arity* — and the single-input designs cannot prove it, because a one-input ROM is precisely the regime most favorable to tabulation (it ties exp). We therefore measure the claim directly on the smallest non-trivial multi-input case: the two-argument EML operator itself, `f(x, y) = exp(x) - ln(y)`.

This needs no new symbolic-regression run. A depth-1 EML tree that routes both external inputs — `x` to the exp leg, `y` to the ln leg — computes exactly `exp(x) - ln(y)` after snapping, with no soft mixing left. The "2-input symbolic unit" is thus the raw gate, and its datapath is just the two already-verified single-input legs (the 257-entry interpolated exp LUT and the range-reduced ln) plus one saturating subtract. Its golden model is `hardware/fixed_point.py::FixedEML.eml_fix`; over a 129×129 grid on the same domains ($x \in [-2, 2]$, $y \in [0.1, 10]$) at a unified Q8.8 it has fp max-error **0.0248** and stores **514 table entries (2 × 257), independent of the input wordlength** (`mlp/symbolic_2in.py`).

The direct ROM for the same function is addressed by *both* input codes, `ROM[{x_code, y_code}]`, so its entry count is the **product** $2^{(w_x + w_y)}$. Sweeping a common per-input wordlength `w_in` (`mlp/rom_baseline.py`, Q8.8 output) makes the contrast quantitative:

unit	entries	EBR	fp max-err	fits iCE40?
symbolic $\exp(x) - \ln(y)$	514 (2×257 , flat in <code>w_in</code>)	—	0.0248	yes
direct ROM, <code>w_in</code> = 4/input	256	1	2.277	yes
direct ROM, <code>w_in</code> = 6/input	4 096	16	0.800	yes (HX8K)
direct ROM, <code>w_in</code> = 8/input	65 536	256	0.0164	no ($8 \times$ HX8K)
direct ROM, <code>w_in</code> = 10/input	1 048 576	4 096	0.0055	no

The pattern is the whole argument. At every ROM resolution that *fits* an iCE40 (`w_in` ≤ 6 , ≤ 32 EBR) the table is $\geq$ **32× less accurate** than the symbolic unit (0.80 vs 0.025); the ROM first matches symbolic accuracy at `w_in` = 8, where it needs **256 EBR — 8× the 32 on an HX8K, infeasible on any iCE40**. The product blows up before the accuracy arrives. The symbolic unit, by contrast, pays for the second input *additively*: its storage is the sum of the two legs (514 entries) and does not move with `w_in` at all. This is the single-input ROM-collapse of §4.3 reproduced one dimension up, and it sharpens with each further input — `k` inputs cost the symbolic datapath $O(\Sigma \text{ legs})$ and the direct ROM $O(n \text{ domains})$.

The synthesized three-corner picture confirms it. All three units pass through the same byte-serial stream wrapper (19 pins) and yosys `synth_ice40` → nextpnr-ice40 UP5K sg48 flow (placement pinned with `--seed 1` for reproducible LC/Fmax), each bit-exact 256/256 in Icarus against its golden model (`mlp/run_2in.py`, `results/mlp_pareto_2in.csv`):

unit	LCs	EBR	Fmax (MHz)	fp max-err	smallest iCE40
symbolic $\exp(x) - \ln(y)$	969	0	12.9	0.0248	HX1K
MLP 2→8→1 (Q8.8 PTQ)	3 536	0	13.7	0.880	UP5K
MLP 2→16→1 (Q8.8 PTQ)	6 497	0	—	0.394	HX8K (overflows UP5K)

The 2-input symbolic unit fits the smallest iCE40 (969/1280 LC on an HX1K) at 0.025 max-error; the 2→8→1 MLP is 3.6× the logic and **35× less accurate**, and the 2→16→1 MLP already overflows the UP5K while still **16× worse** on accuracy. So the single-input dominance result (§4.2) and the ROM-collapse result (§4.3) both survive the step up in arity: against the *learned* baseline the symbolic unit wins on area and accuracy at once, and against the *tabulated* baseline its cost grows additively where the ROM's grows multiplicatively. (The ROM EBR/accuracy rows are synthesis-free — set by the address width — and final; the LC/Fmax rows are placed measurements.)

4.5 On silicon — and a hard realizability gap

The symbolic exp unit runs on a physical iCEstick (HX1K), 256/256 bit-exact (`hardware/host_demo.py`), at 530 LCs (41% of the 1280-LC device).

We attempted the same on-silicon confirmation for the MLP baseline. The machinery is in place — `mlp/board.py` emits an `icestick_mlp_top` bridging the same UART to a chosen exp MLP core — but the baseline does not fit the iCEstick at all. Even the smallest, least accurate exp MLP (1→4→1, 0.40 max-error) places at 1423 ICESTORM_LCs, 111% of the HX1K. Every more-accurate MLP is larger still. On this board the comparison is therefore the sharper "symbolic is the *only* one of the two that is physically realizable." A physical MLP result requires a larger device; we add an iCE40-HX8K Breakout target (7680 LCs, `hardware/hx8k_breakout.pcf`), where the symbolic exp unit places

at 530/7680 LC (6%, 69.7 MHz) and the symbolic In unit fits too (1964 LC / 13 EBR). Only the smaller, lower-accuracy MLPs fit the HX8K; every MLP that approaches the symbolic accuracy overflows even this device. The HX8K, like the HX1K, has no hard multipliers, so MLP MACs still map to fabric LUTs and the area-dominance result is unchanged — the larger device only provides the room to confirm it.

5. Verifiability

The symbolic unit admits an exhaustive correctness statement: over its finite input domain (256 codes on the demo sweep, or the full Q-format domain in `tests/test_hardware.py`), every output equals the fixed-point image of the true function within a stated bound, and the RTL equals that model bit-for-bit. There is no training-noise term. The direct ROM shares this property — the table is the specification. The MLP inherits the same $RTL == model$ guarantee (also 256/256), but " $model == \text{intended function}$ " holds only to the trained approximation error — a qualitatively weaker claim for safety- or certification-sensitive use. This places the symbolic unit and the ROM in one verifiability class and the MLP in another; among the verifiable units, the symbolic one is the realizable one as soon as input width or arity grows (§4.3, §4.4).

6. Discussion

The result has one load-bearing idea and one practical consequence.

The idea: **a snapped symbolic-regression primitive is a compressed ROM that the optimizer discovered rather than the engineer hand-derived**. A direct ROM is the trivially exact unit; range reduction and interpolated LUTs are the standard compressions of it; symbolic regression *recovers* a form that, after snapping, is exactly such a compressed unit, without the engineer specifying the reduction. The three corners — ROM, MLP, symbolic — are then not three unrelated baselines but a single axis: exactness with no compression (ROM), compression with no exactness (MLP), and exact compression (symbolic).

The consequence: on DSP-less fabric the symbolic unit is the only option that is compact, bit-exact, *and* realizable for both functions. The ROM ties it on exp but is unrealizable on In; the MLP is unrealizable at competitive accuracy on either. For edge inference where the nonlinearity must be both small and certifiable, "discover the symbolic form, snap it, compile it" is a viable design flow, and the verifiability argument survives even on a DSP-rich part where the area gap would shrink.

7. Threats to validity / scope

- **Low arity: one 2-input design measured, higher arity extrapolated.** The headline area–accuracy frontier rests on the two single-input designs exp and In — the regime most favorable to the ROM (which ties exp). §4.4 takes one step up in arity and measures the 2-input EML gate end-to-end (placed area, Fmax, bit-exact sim), confirming the symbolic unit beats both the learned and tabulated baselines at arity 2 and that its cost is additive while the ROM's is multiplicative. Higher arities ($k > 2$) are extrapolated from the $O(\Sigma)$ vs $O(\Pi)$ scaling rather than measured, and all designs remain elementwise (no weight sharing across a vector input, where an MLP would amortize MAC cost).
- **iCE40 specifically has no hard multipliers.** On a DSP-rich FPGA the MLP area gap would shrink (the verifiability and exactness arguments are unchanged).
- **MLPs are post-training quantized, not QAT-trained.** As argued in §3, this is not the limiting factor — the float ceiling already lies above the symbolic accuracy — but we report PTQ numbers rather than a QAT sweep.

- **MLPs are small and trained to convergence on a dense grid.** Larger or differently regularized networks could shift the frontier but not cross the symbolic point at the accuracies observed here.

8. Conclusion

A snapped EML expression tree, recovered by symbolic regression, is after argmax commitment a tiny fixed-point function unit. On a DSP-less iCE40 it dominates a quantized-MLP baseline on area and accuracy at once, is bit-exact to the true function (confirmed 256/256 on a physical iCEstick), and against a direct ROM ties for the cheap single-input function exp while remaining realizable for $\ln$, where the ROM needs 512 block RAMs and fits no iCE40. The unifying lens is that the snapped primitive is a compressed ROM — exact like the table, compact like the network — which is precisely why it lands in the realizable gap between them and why its advantage grows with input width and arity. Symbolic-regression discovery is thus a practical route to compact, exhaustively verifiable function units for certification-sensitive edge inference. All claims are reproducible from the scripts in `m1p/` and `hardware/`.

AI Utilization Statement

This statement describes the use of AI systems in producing this work, in keeping with emerging norms for AI disclosure in scientific publishing (cf. ACM, Nature, Science 2024–2025 author guidelines).

The author originated the thesis, supplied source materials, and made all editorial decisions. Claude (Anthropic; Opus and Fable models via the claude.ai web interface and the Claude Code CLI) was used across multiple sessions during the Track C hardware work, including implementation of the fixed-point models, RTL emitters, the MLP and ROM baselines, the 2-input arity experiment (§4.4 — the 2-input symbolic and ROM units, the byte-serial wrapper, the 2-input MLP baseline, and the synthesis sweep with pinned placement), and the drafting of this note. External SME / referee review identified the comparison gaps addressed in §3 (QAT fairness), §4.3 (the direct-ROM baseline), and §4.4 (the arity argument, promoted from a scope caveat to a measured result).

The released code and CSVs are fully reproducible from the released artifacts. No AI system is needed to re-run the experiments or regenerate the figure; the note's empirical claims stand or fall on the released data and synthesis logs alone. AI contribution was to the research process and writing, not to the underlying result.

The human author (Daniyel Yaacov Bilar) takes full responsibility for the scientific content, factual accuracy, and framing of this work. Errors, if any, are the author's.

References

- Bilar, D. Y. (2026). *Valid and False Snapping in EML Expression Trees: The Basin Selection Problem* (v2.3). DOI: 10.5281/zenodo.19790799. Released with `snapping_v2_final.csv` in this repository.
- Bilar, D. Y. (2026). *Basin Selection in EML Expression Trees: Targeted Warm-Start and Top-Aligned Curriculum* (v1.0). DOI: 10.5281/zenodo.20671272.
- Odrzywolek, A. (2026). All elementary functions from a single binary operator. arXiv:2603.21852 (main text and Supplementary Information, especially Table S7).

Data and Code Availability

All code is in the public `em1-ice40-cybenko` repository. Canonical artifacts for this note: `hardware/` (snapped-EML units, fixed-point model, RTL emitters, iCEstick demo), `m1p/` (MLP and ROM baselines, synthesis sweep), `results/m1p_pareto.csv` and `results/m1p_pareto_meta.txt` (synthesis results + tool versions), `results/m1p/train_summary.csv` (per-cell best-of-8 accuracy + spread), and

notes/figure_symbolic_vs_mlp.png.

Exact reproduction commands:

```
python -m mlp.train_mlp          # 8-seed Hxdepth sweep -> results/mlp/ + train_summary.csv
python -m mlp.run_mlp            # emit core + stream RTL for every (best-seed) cell
python -m mlp.sim_check_mlp      # bit-exact 256/256 (Icarus), all cells
python -m mlp.rom_baseline       # direct-ROM spec/error table, 1-D (§4.3) + 2-D arity
sweep (§4.4)
python -m mlp.symbolic_2in       # 2-input symbolic unit exp(x)-ln(y): error report + RTL
 (§4.4)
python -m mlp.run_2in --mlp      # synth half: bit-exact + placed area, symbolic + MLP
 (§4.4)
python -m mlp.synth_sweep        # yosys+nextpnr -> results/mlp_pareto.csv (+ _meta.txt;
incl. ROM rows)
python -m mlp.make_pareto_figure # notes/figure_symbolic_vs_mlp.png
python -m pytest tests/test_mlp_hardware.py -q

# Symbolic units + on-silicon confirmation
python -m hardware.run_csv       # all valid CSV forms -> RTL (collapses to 2 designs)
python -m hardware.sim_check     # bit-exact Icarus vs Python fixed-point model
python -m hardware.build_icestick --flash
python -m hardware.host_demo --port COMx # 256-point sweep, 256/256 bit-exact on silicon
```